
Milestone Report

Spatio-Temporal Video Anomaly Detection using a 3D CNN

Institution	Madhav Institute of Technology and Science (MITS)
Implementation environment	Google Colab with Google Drive persistence
Model approach	PyTorch 3D CNN trained from random initialization
Dataset strategy	UCF-Crime untrimmed surveillance videos with weak video-level labels
Current milestone	Architecture, training plan, evaluation plan, and reproducible Colab workflow

Project objective. Build an intelligent video-analysis pipeline that learns motion and appearance jointly, scores abnormal activity in untrimmed surveillance video, and produces reviewable evidence without relying on pretrained feature extractors.

Prepared from the approved project README and system architecture documents.

1. Executive Summary

Conventional surveillance depends heavily on continuous human observation or frame-by-frame object detection. Those approaches do not reliably capture a behaviour that unfolds across time, such as a fight, accident, theft, or robbery. This capstone addresses that gap with a spatio-temporal anomaly detector that evaluates short video volumes instead of isolated frames.

The project is intentionally scoped for reproducible implementation in Google Colab. A custom PyTorch 3D CNN is trained from randomly initialized weights on UCF-Crime-derived video bags. It outputs an anomaly score for every overlapping 16-frame clip. Multiple-instance learning (MIL) permits training from video-level normal/anomalous labels, while smoothness and sparsity regularization encourage temporally stable and localized predictions.

The milestone deliverable is not a production camera platform. It is a complete training, validation, testing, and offline video-inference workflow that preserves checkpoints, configurations, metrics, score timelines, and annotated MP4 outputs in Google Drive.

2. Problem Statement and Objectives

Objective	Milestone response
Recognize temporal behaviour	Use 3D convolutions over 16-frame clips so motion and appearance are learned together.
Work with long untrimmed video	Treat each video as a bag of clips and train with video-level labels using MIL ranking.
Train from scratch	Implement a custom PyTorch 3D CNN with random initialization; do not use pretrained backbones or extracted C3D features.
Provide reviewable outcomes	Save timestamped score CSVs, score plots, evaluation reports, and annotated MP4 videos.
Maintain reproducibility	Use a deterministic manifest, seeded runs, pinned requirements, saved configuration, and Drive-backed checkpoints.

3. Proposed System Architecture

The architecture separates model development from offline inference. Google Drive is the persistent storage layer, and Google Colab provides the GPU runtime. This avoids dependence on a long-running server while retaining all artefacts needed for a demonstration or result reproduction.

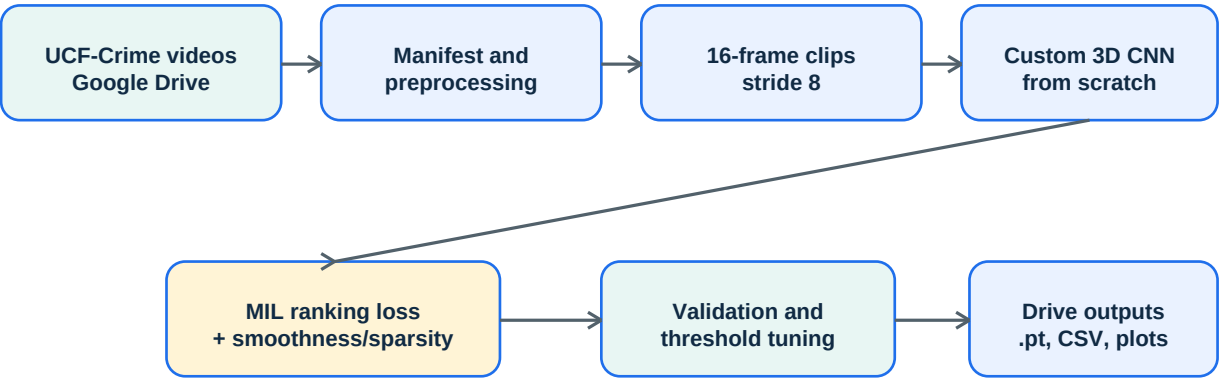


Figure 1. Offline training architecture. The 3D CNN and anomaly head are optimized end-to-end from scratch using clip bags created from untrimmed videos.

Input frames are decoded at a fixed target frame rate, resized to 128 x 171, cropped or resized to 112 x 112, and normalized consistently. Clips have tensor shape [C=3, T=16, H=112, W=112] and use an 8-frame overlap. Timestamp metadata travels with every clip to support temporal evaluation and annotated output generation.

4. Model and Learning Strategy

Layer / mechanism	Design decision	Reason
3D CNN backbone	Custom PyTorch architecture with 3 x 3 x 3 convolution kernels and random initialization	Retains temporal information through the network and fulfills the from-scratch requirement.
Input clip	RGB tensor: 3 x 16 x 112 x 112; stride 8	Balances temporal context, overlap, and Colab compute limits.
Anomaly head	Learned features followed by sigmoid score	Returns one anomaly probability-like score for each clip.
Training bag	All clips from one normal or anomalous video	Uses available video-level labels without requiring expensive segment labels.
Calibration	Validation-set threshold only	Prevents test-set leakage and makes final evaluation credible.

5. MIL Loss and Temporal Decision Logic

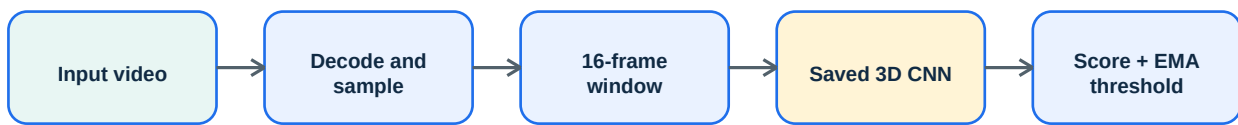
In an anomalous bag A and a normal bag N, the network should rank the highest-scoring clip in A above the highest-scoring clip in N by a margin m. The loss used for the project is:

$$L_{\text{rank}} = \max(0, m - \max(s(A)) + \max(s(N)))$$

The total objective augments the ranking term with sparsity and temporal smoothness:

$$L = L_{\text{rank}} + \text{lambda_sparse} \sum(s(A)) + \text{lambda_smooth} \sum(|s_t - s_{(t-1)}|)$$

Sparsity represents the fact that an anomalous event often occupies only a short part of an untrimmed video. Smoothness reduces unstable score changes between adjacent clips. During video prediction, scores are additionally smoothed with an exponential moving average (EMA). An anomaly interval begins after N consecutive clips exceed a threshold and closes after M clips fall below it.



Outputs: clip-score CSV, score plot, annotated MP4

Figure 2. Offline inference workflow. The same preprocessing module used during training is reused at prediction time.

6. Google Colab Workflow

Notebook	Main responsibility	Persistent output
01_setup.ipynb	Mount Drive, verify GPU, install pinned packages, set seeds and paths.	Environment record and run configuration
02_prepare_data.ipynb	Create deterministic manifests, validate videos, inspect clips and labels.	CSV manifests and quality report
03_train.ipynb	Define the 3D CNN, MIL loss, optimizer, training loop, and checkpoint schedule.	best_model.pt, last_model.pt, logs
04_evaluate.ipynb	Load the frozen best checkpoint and compute held-out metrics.	metrics.json, ROC/PR curves, error analysis
05_predict_video.ipynb	Score an input video and create visual evidence.	prediction CSV, score plot, annotated MP4

Google Drive layout

MyDrive/anomaly_detection/data/ucf_crime - raw videos and manifests

MyDrive/anomaly_detection/checkpoints - best and latest checkpoints

MyDrive/anomaly_detection/runs - logs, curves, and metrics

MyDrive/anomaly_detection/outputs - annotated MP4s and score CSVs

7. Team Responsibilities and Implementation Status

Module	Owner focus	Milestone deliverable
1. Data pipeline	Prepare UCF-Crime manifests, decode/normalize frames, build overlapping clips with timestamps.	Verified data contract and reusable preprocessing functions.
2. 3D CNN training	Implement from-scratch model, MIL loss, validation loop, checkpoints, and metrics.	Training/evaluation notebook design and experiment configuration.
3. Integration and presentation	Maintain shared Colab modules; generate plots and annotated video; document result reproduction.	Demonstration workflow and visual result assets.

At this milestone, the project has a documented end-to-end architecture, model-learning formulation, Colab notebook plan, storage convention, evaluation protocol, and risk controls. Implementation proceeds by first validating the data pipeline on a small approved subset, then training the custom network and finally evaluating a frozen checkpoint on held-out videos.

8. Evaluation and Verification Plan

Gate	Evidence to submit
Dataset integrity	No corrupted videos; deterministic split manifests; no train/validation/test leakage; clip timestamps verified.
Model quality	Validation ROC-AUC, PR-AUC, false alarms per video/hour, time-to-detect, and category-wise analysis.
Threshold integrity	Threshold selected on validation data and frozen before final test evaluation.
Reproducibility	Saved seed, configuration, manifest version, checkpoint hash, package versions, and repeated evaluation.
Inference output	Timestamp-aligned prediction CSV, readable score plot, and playable annotated MP4.
Failure analysis	Representative low-light, crowding, occlusion, camera-shake, and unseen-normal cases.

9. Risks and Mitigations

Risk	Impact	Mitigation
Class imbalance and novel anomalies	Missed anomalies or false positives	Use normal and anomalous bags, report per-category performance, and retain uncertain examples for analysis.
Darkness, occlusion, crowding, camera shake	Scores may reflect video quality rather than activity	Include failure-case review and diverse validation examples; do not overstate automated decisions.
Colab session resets	Loss of progress	Save checkpoints, configuration, and logs to Drive at every epoch; resume from latest checkpoint.
Limited GPU time	Slow experimentation	Debug on a small approved subset, use efficient input settings, mixed precision where supported, and resumable runs.
Inconsistent transforms	Unreliable test results	Centralize preprocessing and record transforms in <code>run_config.json</code> .

10. Scope, Ethics, and Next Milestone

The model is a decision-support system. It flags unusual activity for review; it does not prove criminal intent, identify people, or replace trained security personnel. In particular, anomaly labels must be interpreted carefully because rare behaviour can be harmless and surveillance footage may be biased by lighting, camera location, and data collection practices.

Next milestone activity	Expected artefact
Implement and smoke-test preprocessing on a small subset	Manifest, sampled clip visualizations, and shape/timestamp tests
Train baseline custom 3D CNN	Checkpoint, learning curves, and validation report
Tune threshold and evaluate frozen model	Test metrics, ROC/PR curves, false-alarm analysis
Demonstrate offline inference	Annotated UCF-Crime/user video and score timeline

Conclusion

The approved design offers a technically grounded route from untrimmed surveillance video to explainable, timestamped anomaly evidence. The immediate focus is a reproducible, from-scratch 3D CNN implementation in Google Colab, with carefully controlled training, validation, and evaluation rather than premature real-time deployment claims.

11. Research Basis

Reference	Contribution to this project
Tran et al., Learning Spatiotemporal Features with 3D Convolutional Networks	Supports 3D convolutions, $3 \times 3 \times 3$ kernels, 16-frame clips, and overlapping temporal feature extraction.
Sultani, Chen, and Shah, Real-world Anomaly Detection in Surveillance Videos	Supports UCF-Crime, weak video-level supervision, MIL ranking, temporal smoothness, and sparsity constraints.
Pang et al., Deep Learning for Anomaly Detection: A Review	Highlights rare and heterogeneous anomalies, class imbalance, false positives, novelty, and the need for human review.